

Rezumat — Această lucrare investighează integrarea inteligenței artificiale (IA) în ingineria software, concentrându-se pe aplicabilitatea sa practică în dezvoltarea sistemelor moderne. În prima parte sunt analizate contribuțiile IA în automatizarea etapelor ingineresti, incluzând generarea de cod, testarea continuă și asistarea decizională. Sunt identificate beneficiile majore, dar și limitările curente, cum ar fi lipsa explicabilității, dependența de structuri formale și riscurile asociate validării codului generat automat. A doua parte documentează un studiu de caz centrat pe aplicația „ExamPlanner” – o soluție digitală pentru programarea examenelor universitare – dezvoltată prin colaborarea umană și asistarea AI. Sunt evaluate performanțele și gradul de integrare a mai multor utilitare AI: ChatGPT, BlackBox AI, WindSurf, CiteSpace, ClickUp și DrawIO. Analiza evidențiază faptul că IA poate accelera dezvoltarea și îmbunătăți calitatea aplicațiilor doar atunci când este susținută de o arhitectură logică solidă și o viziune inginerască clară.

Cuvinte cheie — *Inteligență artificială, inginerie software, ChatGPT, automatizare, dezvoltare software, validare cod, proiect management asistat de AI, DrawIO*

I. FUNDAMENTE TEORETICE PRIVIND IA ÎN INGINERIA SOFTWARE

A. Introducere

Inteligența artificială (IA) și ingineria software reprezintă astăzi două domenii profund interconectate, aflate în centrul transformărilor digitale din toate sectoarele. În contextul dezvoltării de aplicații moderne – precum platformele dedicate programării automate a examenelor universitare – utilizarea instrumentelor bazate pe IA nu mai este opțională, ci devine o componentă strategică pentru eficiență, personalizare și scalabilitate.

Literatura de specialitate evidențiază faptul că sistemele software cu componente IA (AI-based systems) prezintă particularități distincte față de aplicațiile tradiționale: acestea învață din date, iau decizii pe baza unui context evolutiv și implică colaborare interdisciplinară între ingineri software, analiști de date și experți de domeniu [1]. Dezvoltarea unor astfel de sisteme implică metodologii specifice de testare, validare și mentenanță, având în vedere caracterul imprevizibil al

comportamentului algoritmic și cerința de adaptabilitate continuă [1].

De asemenea, cercetările evidențiază modul în care IA contribuie la îmbunătățirea proceselor decizionale și automatizarea sarcinilor repetitive în aplicațiile software. Spre exemplu, utilizarea chatbot-urilor bazate pe IA sau a analizelor predictive a devenit tot mai frecventă în aplicațiile educaționale și administrative [2]. Algoritmii inteligenți permit formularea de recomandări personalizate, detecția automată a anomaliilor și analizarea rapidă a volumelor mari de date, ceea ce conduce la creșterea accesibilității și a securității digitale [2].

Lucrarea este structurată în două direcții complementare. Prima parte este dedicată analizei literaturii de specialitate axate pe intersecția dintre AI și ingineria software, cu accent pe surse academice publicate în reviste precum IEEE Software, ACM TOSEM sau Finance & Accounting Research Journal. Cea de-a doua parte constă într-un studiu de caz aplicat, care documentează experiența utilizării unor instrumente AI – inclusiv ChatGPT, Windsurf și Blackbox AI – în procesul de concepere și dezvoltare a unei aplicații pentru programarea examenelor.

B. Inteligența Artificială în contextul dezvoltării software

Aplicațiile bazate pe inteligență artificială contribuie semnificativ la automatizarea etapelor de planificare și implementare software, prin utilizarea unor instrumente care permit conversia limbajului natural în cod sursă și susțin procesele de depanare și compilare automată [3]. Această automatizare oferă dezvoltatorilor posibilitatea de a aloca mai mult timp activităților creative, precum proiectarea arhitecturii aplicației sau optimizarea funcțională a componentelor.

Beneficiile IA sunt vizibile și în ceea ce privește integrarea sistemelor, testarea continuă și detectarea timpurie a erorilor. Algoritmii inteligenți pot anticipa defecte din fazele incipiente ale dezvoltării software, contribuind astfel la reducerea costurilor și a efortului uman implicat în procesele de mentenanță [3].

În ciuda acestor avantaje, rolul IA rămâne, în forma actuală, predominant suportiv. Generarea automată de cod sau

identificarea tiparelor în date nu pot înlocui procesele decizionale complexe și definirea specificațiilor sistemului, care necesită intervenție umană calificată [3]. Expertiza inginerescă rămâne crucială în stabilirea cerințelor funcționale și în adaptarea aplicațiilor la contexte specifice.

În acest cadru, se evidențiază și o transformare a rolului tradițional al inginerului software, care trebuie să dezvolte competențe suplimentare. Pe lângă abilitățile tehnice de bază, este necesară o bună înțelegere a mecanismelor IA, precum și capacitatea de a colabora eficient cu sisteme semi-autonome. Creativitatea, capacitatea de abstractizare multiplă și gândirea originală devin competențe esențiale în acest nou ecosistem tehnologic.

Deși literatura actuală susține potențialul ridicat al IA în automatizarea sarcinilor ingineresti și valorificarea masivă a datelor, aceste sisteme rămân dependente de arhitecturi clar definite și constrângeri formale. Dezvoltarea unor soluții complet autonome și adaptabile rămâne în continuare o direcție de cercetare deschisă și provocatoare [3].

C. Analiza literaturii de specialitate

Literatura recentă consolidează o perspectivă din ce în ce mai detaliată asupra modului în care inteligența artificială se integrează în activitățile ingineresti din domeniul dezvoltării software. Utilizarea modelelor generative, inclusiv a celor de tip large language models (LLM), influențează în mod direct fluxurile de dezvoltare, afectând procese precum definirea cerințelor, redactarea documentației, revizuirea codului și colaborarea în cadrul echipelor tehnice [4]. Aceste tehnologii determină nu doar o adaptare procedurală, ci și o reconfigurare a responsabilităților și culturii organizaționale în care sunt implementate.

Integrarea IA în aplicații critice este reflectată și în soluțiile software dezvoltate pentru siguranța la incendii, unde algoritmi inteligenți sunt utilizați pentru simularea și proiectarea sistemelor de protecție [5]. Antrenarea modelelor pe seturi de date istorice, validarea acestora de către specialiști și includerea într-un flux ingineresc coerent confirmă aplicabilitatea IA în contexte în care precizia și robustețea sunt esențiale.

Direcțiile viitoare ale industriei software sunt, la rândul lor, condiționate de nivelul de integrare al IA și de autonomia echipelor de dezvoltare. În unele scenarii avansate, scrierea manuală de cod este redusă semnificativ, iar rolul dezvoltatorului este reconstruit în jurul coordonării și validării soluțiilor generate [6]. Aceste proiecții ridică însă probleme legate de transparență, supradependență și necesitatea păstrării controlului uman.

În acest context, dificultatea evaluării rezultatelor generate automat devine un obstacol relevant. Lipsa explicabilității deciziilor IA, alături de barierele cognitive și organizaționale, impune dezvoltarea unor abordări hibride care să mențină componenta umană în centrul procesului decizional [7].

Pe plan tehnologic, se observă o reconfigurare profundă a ingineriei datelor. Infrastructura de dezvoltare devine dinamică, iar colectarea, etichetarea și guvernarea datelor influențează direct calitatea aplicațiilor software, în special în arhitecturi CI/CD și sisteme de învățare continuă [8].

Toate aceste direcții confirmă că IA nu funcționează ca un instrument auxiliar în ingineria software, ci ca un catalizator al transformării proceselor. Totuși, implementarea acestor tehnologii rămâne neuniformă și este însoțită de provocări semnificative, precum lipsa transparenței decizionale, dificultatea integrării în structuri organizaționale și nevoia de reconfigurare a paradigmatelor cognitive. Doar printr-o abordare multidisciplinară, responsabilă și adaptativă, IA poate fi valorificată sustenabil în practici ingineresti contemporane.

D. Provocări și limitări

În ciuda progreselor remarcabile în integrarea IA în ingineria software, literatura de specialitate conturează o serie de limitări structurale, tehnologice și epistemologice care trebuie abordate critic. În primul rând, provocările apar încă din etapa de planificare a proiectelor, unde, în ciuda automatizării bazate pe algoritmi evolutivi sau căutare euristică, complexitatea straturilor decizionale și incertitudinile dinamice pun în dificultate optimizarea reală a resurselor și a cerințelor clientului [3].

Pe plan metodologic, IA nu este capabilă să ofere soluții complet autonome, întrucât creativitatea și înțelegerea contextuală rămân apanajul dezvoltatorilor umani. Interviuurile calitative realizate în cadrul studiilor recente arată că dezvoltatorii văd în IA un instrument de sprijin mai degrabă decât un partener inovativ, accentuând astfel rolul critic al colaborării om-mașină în procesele de design și mentenanță software.

O limitare semnificativă este asociată validării și testării codului generat. Progresele în generative IA complică identificarea sursei originale a codului, iar riscurile cibernetice cresc exponențial din cauza posibilității inserării de fragmente de cod malițios greu de detectat. Astfel, verificarea și validarea rezultatelor IA devin aspecte esențiale în menținerea unui nivel acceptabil de siguranță și încredere.

În ceea ce privește procesul de testare software, inteligența artificială întâmpină dificultăți legate de lipsa unor date suficiente sau reprezentative pentru antrenare, ceea ce conduce la modele cu performanțe neuniforme și greu de reprodus. Aplicațiile critice, cum ar fi cele din domeniul medical sau al transportului autonom, sunt deosebit de vulnerabile la aceste limite, întrucât datele incomplete pot genera comportamente imprevizibile ale sistemului.

Structura sistemelor IA este, de asemenea, o sursă de complexitate. Modelele de învățare automată implică interdependențe greu de controlat (CACE – „Changing Anything Changes Everything”), iar arhitecturile software care integrează componente multiple IA și non-IA devin greu de întreținut și validat. Acest fenomen conduce la apariția unor „anti-paternuri” arhitecturale – un indiciu al unei integrări deficitare a IA în procesele ingineresti [1].

Mai mult, IA suferă de o lipsă de transparență în interpretarea deciziilor, cunoscută sub numele de „problema oracolului”. Din această cauză, verificarea funcțională a sistemelor bazate pe IA este afectată de lipsa unei „realități de referință” universal acceptate, ceea ce complică definirea scenariilor de testare și evaluarea corectitudinii rezultatelor obținute [1].

În final, se remarcă faptul că utilizarea IA în dezvoltarea software nu poate înlocui complet procesele tradiționale fără riscuri semnificative. Codul generat automat poate fi incomplet, incorect sau periculos, în lipsa unei supravegheri umane riguroase. Lipsa unei infrastructuri adecvate, a unor metode clare de documentare și trasabilitate și a unor mecanisme robuste de audit contribuie la perpetuarea unei stări de incertitudine în ceea ce privește adoptarea IA la scară industrială.

II. STUDIU DE CAZ: APLICAȚIA „EXAMPLANNER”

A. Prezentare și scopul aplicației

Aplicația „ExamPlanner” este o soluție digitală destinată automatizării și eficientizării procesului de programare a examenelor în mediul universitar. Aceasta facilitează colaborarea între patru tipuri de utilizatori: administratorul aplicației, secretarul, șeful de grupă și profesorul, fiecare având responsabilități clar definite în cadrul unui flux logic coordonat.

Procesul începe cu administratorul, care se autentifică printr-un cont local generat la fiecare pornire a aplicației. Acesta gestionează implementarea aplicației și de importul inițial de date în format .json, incluzând: conturile secretarilor, grupele academice, sălile de examen, disciplinele și cadrele didactice asociate.

Ulterior, secretarii își accesează contul prin autentificare cu adresa instituțională @usm.ro. Ei introduc în sistem șefii de grupă, creează instanțele de examene pentru fiecare grupă și disciplină și transmit automat prin aplicație emailuri de notificare către șefii de grupă, informându-i despre disponibilitatea selectării datelor de examen. După validarea fluxului, confirmarea finală a listei de examene și exportul în format Excel sau PDF sunt ultimele sarcini ale secretariatului.

Șeful de grupă, autentificat prin contul său @student.usv.ro, accesează instanțele disponibile și selectează data, ora și sala pentru fiecare examen. Aceste propuneri sunt trimise spre validare profesorului aferent.

Profesorul este notificat automat prin email. După autentificare (@usm.ro), acesta poate accepta propunerea sau o poate modifica (data, ora, sala). În cazul unei modificări, propunerea este returnată șefului de grupă pentru o aprobare finală. Profesorul este responsabil și cu selectarea sălii și a unui eventual asistent, iar sala selectată este blocată automat pentru 10 minute, pentru a evita suprapunerile.

După acest schimb iterativ, secretariatul validează planificarea finală, o poate imprima sau exporta, asigurând astfel finalizarea procesului de programare.

B. Utilitare testate

Pe parcursul dezvoltării aplicației, am experimentat și testat mai multe instrumente de tip AI sau asistate de AI, cu scopul de a eficientiza procesele de concepere, documentare, testare și validare. Fiecare dintre acestea a oferit avantaje distincte, dar și limitări notabile care trebuie recunoscute în mod obiectiv. Analiza de mai jos reflectă atât performanțele generale ale fiecărui utilitar, cât și o perspectivă personală asupra utilității și a gradului de integrare în procesul de dezvoltare.

1) ChatGPT

ChatGPT a fost cel mai frecvent instrument utilizat, fiind integrat în toate etapele proiectului, de la formularea cerințelor, arhitectură, corectarea secvențelor de cod și a erorilor, până la redactarea documentației și generarea diagramelor.

În etapa incipientă, ChatGPT a fost utilizat pentru conturarea cerințelor funcționale și pentru definirea cazurilor de utilizare (figura 1). Actorii implicați – administratorul, secretarul, profesorul și șeful de grupă – au fost identificați împreună cu sistemul, iar pentru fiecare s-au dezvoltat cazuri de utilizare în format textual, apoi transpuse în diagrame ASCII. Este remarcabil faptul că modelul a permis corecturi iterative, în funcție de feedbackul oferit, respectând toate cerințele specifice (de exemplu: reprezentarea gestiunii sălilor, blocarea acestora pentru 10 minute, notificarea șefilor de grupă prin email etc.), vezi figura 2.

Un exemplu concludent este generarea diagramei de cazuri de utilizare în format ASCII – o reprezentare detaliată, centrată pe funcționalități, care a trecut prin mai multe revizii succesive. ChatGPT a reușit să redea în mod coerent structura aplicației, adaptând diagramele pentru terminal și optimizându-le vizual, în ciuda constrângerilor de spațiere impuse de fonturile monospațiate. De asemenea, modelul a furnizat automat variante în limba română, a corectat aliniamentele și a ținut cont de formatarea în stil ingineresc.

Tot cu ajutorul ChatGPT au fost dezvoltate și alte tipuri de diagrame UML în ASCII – inclusiv diagrama de clase, diagrama de componente, diagrama de activități și cea de stări – toate însoțite de explicații și propuneri de îmbunătățire. În special pentru partea vizuală, s-a colaborat în mod constant pentru a asigura lizibilitatea și reprezentarea fidelă a logicii aplicației.

Un aspect remarcabil a fost capacitatea modelului de a menține contextul conversației pe termen lung. De-a lungul a zeci de interacțiuni, a reușit să revină la concepte discutate anterior, să sugereze îmbunătățiri pe baza versiunilor anterioare și să asigure coerența între părțile componente (de exemplu, între cazurile de utilizare și diagrama de activități, sau între fluxul decizional și diagrama de stări).

De asemenea, ChatGPT a fost utilizat în redactarea documentației și referatului teoretic, mai ales în prima parte. Modelul a fost capabil să extragă și să sintetizeze idei din articole științifice în format PDF, integrând corect sursele, evitând exprimările hiperbolice și păstrând un ton academic echilibrat, conform solicitării.

Ca puncte forte, se remarcă:

- adaptabilitatea la cerințe complexe și detaliate;
- capacitatea de a structura logic conținutul și de a menține coerența globală;
- viteza de răspuns și disponibilitatea pentru revizuii rapide.

Totuși, au existat și limitări:

- dificultăți în generarea automată a diagramelor în format imagine conform așteptărilor stilistice, vezi figura 3;

- unele repetări în exprimări, dacă nu se furnizează indicații clare de evitare;
- necesitatea unei verificări finale asupra conținutului tehnic, adeseori formulat ambiguu.



Figura 1) Secvența de generare a diagramei de cazuri

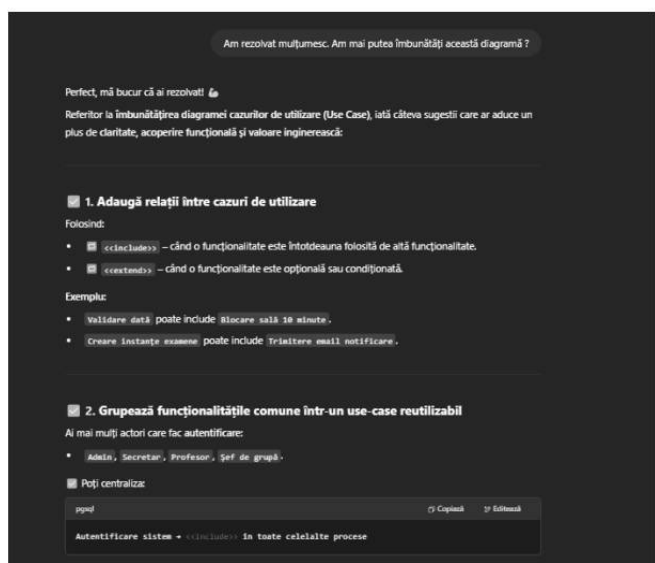


Figura 2) Corectarea diagramei de cazuri

În ansamblu, ChatGPT a funcționat ca un partener de lucru eficient, oferind suport în toate etapele dezvoltării aplicației ExamPlanner, de la concept până la redactarea lucrării academice. Fără îndoială, un astfel de instrument va rămâne parte integrantă în proiectele viitoare, atât pentru prototipare și design, cât și pentru redactare și validare conceptuală.

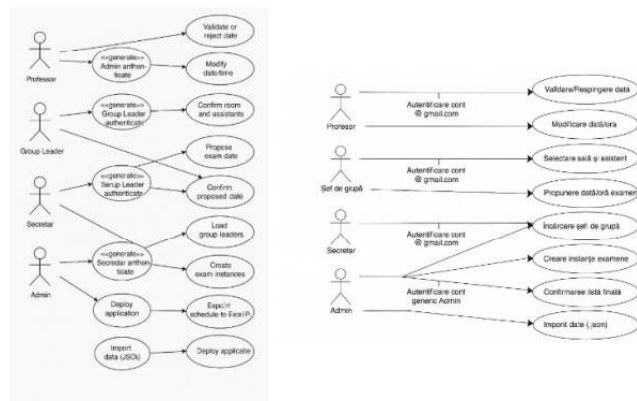


Figura 3) Generare automată diagramare sub formă de imagine

2) BlackBox AI

Blackbox AI a fost utilizat în mod punctual, cu un scop bine delimitat: acela de a genera și compara soluții de cod sursă pentru sarcini specifice din cadrul aplicației ExamPlanner. Nu a fost ales ca instrument principal de dezvoltare, ci mai degrabă ca o unealtă de referință, pentru a evalua și valida alternative la soluțiile propuse de ChatGPT, în special în situațiile în care codul trebuia să fie sintactic curat și logic simplu.

În general, interacțiunea cu Blackbox AI s-a dovedit utilă în taskuri clar definite – precum generarea de funcții simple pentru conversia datelor, manipularea stringurilor în FastAPI sau scrierea unor funcții Python pentru serializare JSON. Spre deosebire de alte unelte bazate pe AI, Blackbox a oferit de cele mai multe ori rezultate direct funcționale, fără erori de compilare sau neclarități sintactice, dar cu un nivel redus de detalii explicative.

Un aspect pozitiv a fost concizia. Codul propus de Blackbox a fost „aerisit”, cu o structură minimalistă, ideal pentru sarcini unde claritatea era esențială. Spre exemplu, pentru generarea unui endpoint simplu în FastAPI sau pentru configurarea unui model Pydantic, rezultatele oferite au fost aproape întotdeauna corecte și eficiente. În acest sens, Blackbox s-a dovedit a fi un aliat de încredere pentru verificări rapide, fără a încălca fluxul de lucru.

Totuși, spre deosebire de ChatGPT, Blackbox nu a oferit suport explicativ sau argumentativ – a fost lipsit de capacitatea de a justifica deciziile de proiectare sau de a adapta soluția la un context mai larg al aplicației. De asemenea, în taskuri mai complexe – cum ar fi manipularea datelor între mai multe servicii sau lucrul cu JWT tokens în contextul Firebase – abordările sale au fost adesea generice, fără integrare contextuală.

Din punct de vedere al experienței personale, utilizarea Blackbox a fost cel mai valoros exercițiu de validare. În mod repetat, codul oferit a fost comparat cu soluțiile generate de ChatGPT și adaptat acolo unde era necesar un stil mai scurt și mai direct. În acest sens, Blackbox a acționat ca un „barometru” al clarității și minimalismului.

Dacă am mai folosi acest utilitar? Cu siguranță, da – dar într-un mod complementar. Pentru sarcini scurte, bine definite, Blackbox este eficient și rapid. Totuși, pentru elaborarea logicii aplicației, structurarea componentelor sau clarificarea

funcționalităților, alte unelte AI mai „conversationale” rămân superioare.

3) WindSurf

Windsurf reprezintă o unealtă de inteligență artificială emergentă, orientată către dezvoltarea rapidă de aplicații web prin generare automată de cod pe baza unei descrieri inițiale structurate. Spre deosebire de alte instrumente AI centrate pe generarea de secvențe izolate de cod sau explicații, Windsurf urmărește realizarea unei aplicații complete, de la backend la interfață, într-un mod integrat și automatizat.

În procesul de dezvoltare al aplicației ExamPlanner, Windsurf a fost testat ca soluție de prototipare accelerată. În situațiile în care arhitectura sistemului era deja stabilită – cu diagrame de clase și cazuri de utilizare bine definite – Windsurf a reușit să genereze o bază de cod funcțională, incluzând serviciile, modelele de date și chiar structura controllerelor. Această capacitate de a transforma modele abstracte în cod executabil este deosebit de valoroasă în fazele inițiale de dezvoltare, când timpul de reacție este critic, iar iterațiile sunt frecvente.

Totuși, experiența personală cu Windsurf a fost una duală. Pe de o parte, am apreciat viteza de generare și nivelul de integrare oferit în proiectele standardizate. Pe de altă parte, am întâmpinat mai multe limite importante în personalizarea aplicației. Windsurf funcționează optim doar atunci când structura aplicației este clară și complet definită în prealabil – inclusiv relațiile dintre entități și așteptările legate de comportamentul sistemului. În lipsa acestor definiții, codul generat este adesea generic sau incomplet, generând erori. Totodată depanarea erorilor este foarte îngreunată întrucât trebuie foarte bine analizată structura codului deja creat.

Un alt aspect relevant este limitarea accesului la funcționalitățile avansate. Versiunea gratuită a platformei permite doar proiecte simple, iar pentru aplicațiile complexe – așa cum este cazul ExamPlanner – este necesară achiziționarea unei licențe comerciale. Aceasta limitează adoptarea sa în contextul educațional sau în proiectele cu buget restrâns, mai ales atunci când este necesară integrarea cu servicii externe, autentificare avansată sau logici de business personalizate.

Aș mai utiliza Windsurf? Cu siguranță, da – dar într-un cadru bine pregătit dinainte. Dacă documentația sistemului este completă (diagrama de clase, fluxuri, cazuri de utilizare), Windsurf poate economisi semnificativ timp de dezvoltare. În schimb, în proiecte exploratorii sau cu cerințe fluide, dependența de constrângeri stricte reduce utilitatea sa reală.

4) CiteSpace

CiteSpace este recunoscut în mediul academic drept unul dintre cele mai avansate instrumente pentru analiza vizuală a literaturii științifice. Capacitatea sa de a identifica tendințe emergente, rețele de citare, clustere tematiche și autori influenți îl transformă într-o unealtă esențială în domeniul scientometriei și al cercetării interdisciplinare.

În contextul realizării documentației pentru aplicația ExamPlanner, CiteSpace a fost testat ca parte a demersului de a înțelege poziționarea temei (utilizarea AI în ingineria software) în raport cu literatura de specialitate recentă. Potențialul său

analitic s-a dovedit impresionant: dintr-un set de articole PDF, instrumentul poate extrage metadate, construi hărți de co-citare și evidenția conexiuni conceptuale greu de identificat prin lectură tradițională.

Totuși, din perspectiva experienței personale, interacțiunea cu CiteSpace a fost marcată de o curbă de învățare abruptă. Interfața, deși funcțională, nu este una intuitivă, iar documentația necesită o înțelegere prealabilă a conceptelor scientometrice și bibliometrice (ex. betweenness centrality, burst detection, co-word analysis). Fără o instruire teoretică minimă, utilizatorul riscă să folosească parțial sau incorect funcțiile disponibile, ceea ce limitează semnificativ beneficiile oferite de platformă.

În cazul aplicației ExamPlanner, CiteSpace nu a fost folosit ca instrument principal, ci mai degrabă ca o tentativă de explorare a unui AI orientat spre analiza bibliografică. Din această experiență s-a desprins o concluzie importantă: CiteSpace este un instrument puternic, dar nu generalist. Pentru a fi valorificat eficient, este nevoie fie de o instruire dedicată, fie de colaborarea cu persoane familiarizate cu metodologia scientometrică.

Aș mai folosi acest instrument? Da, însă doar într-un context care justifică în mod clar nevoia unei analize științifice avansate – de exemplu, pentru fundamentarea unei teze de doctorat sau realizarea unei cercetări sistematice. În cazul aplicațiilor software cu caracter practic (cum este ExamPlanner), valoarea sa este mai degrabă complementară decât centrală.

5) ClickUP

ClickUp s-a remarcat ca un instrument versatil de proiect management, capabil să integreze într-un singur spațiu de lucru toate etapele ciclului de viață al aplicației – de la planificare și analiză până la livrare și documentare. Folosirea acestui instrument nu s-a limitat la o simplă structurare a sarcinilor, ci a reprezentat un pas esențial în automatizarea fluxului de dezvoltare și sincronizarea echipei în timp real.

Una dintre cele mai valoroase funcționalități a fost posibilitatea de a defini automatizări bazate pe reguli logice – spre exemplu, mutarea automată a sarcinilor între coloanele Kanban („În lucru”, „Verificare”, „Finalizat”) în funcție de statusul definit de utilizator. Aceste automatizări au redus necesitatea unor acțiuni repetitive și au menținut actualizată structura generală a proiectului fără intervenții manuale constante. De asemenea, utilizarea șabloanelor pentru definirea etapelor unui proiect software (analiză, design, implementare, testare, lansare) a permis o planificare coerentă și reutilizabilă în mai multe instanțe.

ClickUp a facilitat și documentarea tehnică și decizională, permițând integrarea de fișiere externe (PDF, diagrame DrawIO, exporturi din Swagger sau Javadoc), precum și comentarii și note interne direct asociate fiecărei sarcini. Astfel, spațiul de lucru a devenit nu doar un panou de management al activităților, ci și un depozit de cunoștințe utile pe termen lung. Funcția de timeline (similară unei diagrame Gantt) a fost utilizată pentru a urmări progresul în timp și a evalua dependențele critice între sarcini, evidențiind blocajele ce puteau apărea între dezvoltarea backend-ului și cea a interfeței grafice.

Totuși, în ciuda acestor beneficii, platforma nu este lipsită de limitări. Automatizările mai complexe – cum ar fi integrarea directă cu fluxuri CI/CD sau generarea de documente tehnice dinamice – necesită planuri comerciale avansate sau dezvoltarea unor interconectări API externe. În plus, lipsa unui editor UML nativ sau a suportului complet pentru diacritice în toate modurile de afișare a creat ocazional mici disfuncționalități în prezentarea documentelor.

Pe ansamblu, ClickUp a funcționat ca un mediu colaborativ eficient și ca un motor de automatizare a fluxului de lucru, contribuind semnificativ la eficientizarea procesului de dezvoltare. În contextul aplicației ExamPlanner, rolul său nu a fost doar organizatoric, ci și strategic, oferind suport continuu în gestionarea resurselor, prioritizarea livrabilelor și facilitarea comunicării între membrii implicați în proiect. Aceste aspecte confirmă faptul că utilizarea platformelor inteligente de management contribuie din ce în ce mai mult la succesul inițiativelor educaționale și software contemporane.

6) DrawIO

DrawIO constituie o platformă digitală avansată pentru realizarea diagramelor, ale cărei funcționalități integrate de inteligență artificială o transformă într-un instrument esențial în procesul de proiectare conceptuală și logică a bazelor de date. În cadrul dezvoltării sistemului informatic destinat planificării examenelor și colocviilor, această aplicație a fost utilizată cu precădere pentru modelarea relațională a datelor, asigurând coerența și lizibilitatea arhitecturii informaționale. Printre funcțiile asistate de AI se numără alinierea automată a entităților, recunoașterea contextuală a relațiilor dintre componente și optimizarea spațială a diagramei, contribuind astfel la o experiență de utilizare intuitivă și la o reducere considerabilă a timpului necesar pentru realizarea unui model robust.

Prin intermediul DrawIO a fost posibilă nu doar reprezentarea vizuală a tabelelor și relațiilor dintre acestea, ci și validarea logicii de ansamblu a bazei de date, facilitând comunicarea inter-disciplinară între dezvoltatorii tehnici și actorii non-tehnici implicați în proiect. Capacitatea platformei de a integra sugestii automate și de a menține consistența grafică între componente s-a dovedit extrem de utilă în generarea unor diagrame clare, profesioniste, conforme cu standardele ingineresti. Astfel, DrawIO a contribuit semnificativ la etapa de analiză și design al sistemului, confirmându-și relevanța în contextul dezvoltării de aplicații educaționale cu structură modulară și complexitate ridicată.

C. Analiza comparativă

Integrarea inteligenței artificiale în procesul de dezvoltare software, în special în cadrul aplicației ExamPlanner, a evidențiat un fapt esențial: AI este un instrument de suport extrem de valoros, dar care nu poate substitui gândirea critică, arhitectura logică și claritatea intențiilor programatorului. Pe măsură ce dezvoltarea a progresat, a devenit tot mai clar că eficiența utilizării acestor unelte depinde în mod direct de capacitatea dezvoltatorului de a defini corect structura, scopul și fluxul de lucru al aplicației.

ChatGPT a reprezentat elementul central al interacțiunii dintre gândirea inginerescă și automatizare. A fost utilizat

pentru generarea de cod, explicarea unor concepte, redactarea documentației și chiar validarea unor abordări logice. În special în scrierea de cod pentru funcționalități bine definite sau în clarificarea unor erori sintactice, ChatGPT s-a dovedit eficient. Totuși, când solicitările erau neclare sau fragmentate, rezultatele generate erau adesea generice sau incomplete. Astfel, a devenit evident că valoarea adăugată de acest instrument depinde de precizia și coerența cu care programatorul formulează cerințele – un dialog clar și contextualizat produce rezultate utile, altfel AI-ul poate doar aproxima intenția.

ClickUp, în schimb, a oferit o abordare diferită, axată pe organizare, urmărirea sarcinilor și documentarea progresului. Deși nu este un AI în sensul clasic, platforma beneficiază de mecanisme inteligente de automatizare și corelare a activităților. Prin utilizarea sa, fluxul de dezvoltare a fost vizibil mai bine organizat: fiecare etapă a fost trasată, planificată și corelată cu celelalte componente, iar instrumentele de analiză vizuală au permis identificarea blocajelor și gestionarea eficientă a timpului. Totuși, puterea sa nu vine din „inteligență” propriu-zisă, ci din flexibilitatea și interconectarea modulelor. A funcționat mai degrabă ca un suport strategic pentru management decât ca un motor de generare automată.

WindSurf, deși promițător prin capacitatea sa declarată de a genera aplicații complete pe baza descrierilor oferite, s-a dovedit în practică dificil de utilizat în lipsa unei structuri predefinite foarte riguroase. Pentru a obține rezultate relevante, a fost necesară furnizarea unor diagrame UML detaliate, cu relații clare între entități, ceea ce, practic, presupunea deja un nivel avansat de analiză și design finalizat în prealabil. Accesul limitat la funcționalități complexe, în lipsa unui cont premium, a îngreunat testarea în profunzime a platformei. Astfel, contribuția sa a fost mai degrabă orientativă decât productivă.

În concluzie, utilizarea AI în dezvoltarea aplicației ExamPlanner a evidențiat faptul că aceste instrumente devin cu adevărat utile doar atunci când sunt susținute de o bază conceptuală solidă. Inteligența artificială, fie că este utilizată pentru generare de cod, automatizare de procese sau asistență organizațională, nu înlocuiește necesitatea unei analize ingineresti riguroase. Ideile de bază, formatul aplicației, cazurile de utilizare și secvențele de interacțiune trebuie stabilite în mod clar de către dezvoltator. AI-ul nu poate deduce intenții nedefinite sau logici incomplete. Astfel, valoarea acestor unelte constă în capacitatea de a accelera implementarea unor soluții deja bine concepute, de a reduce volumul muncii repetitive și de a sprijini procesul decizional – nu de a substitui analiza inginerescă propriu-zisă.

III. CONCLUZII

Rezultatele evidențiate de analiza teoretică și studiul de caz asupra aplicației „ExamPlanner” confirmă faptul că inteligența artificială nu poate fi tratată ca o soluție completă sau autonomă pentru dezvoltarea software, ci mai degrabă ca un instrument sofisticat de asistare, augmentare și accelerare a activităților ingineresti. Eficiența reală a sistemelor AI apare doar atunci când acestea sunt integrate într-un cadru logic bine structurat, în care fluxurile de date, interacțiunile și arhitectura aplicației sunt clar definite de către dezvoltator.

Utilizarea ChatGPT în cadrul proiectului a demonstrat că, în ciuda abilității de a genera cod și documentație coerentă, valoarea adăugată depinde direct de calitatea și claritatea cerințelor formulate. AI-ul poate completa, optimiza sau rafina soluții existente, însă nu este capabil să înlocuiască judecata inginerescă, care rămâne esențială în procesul de luare a deciziilor critice, în special în fazele de proiectare și integrare. ChatGPT s-a remarcat în special prin capacitatea sa de a menține contextul pe termen lung, de a genera rapid prototipuri și de a oferi suport în redactarea documentației tehnice și științifice.

BlackBox AI, deși limitat la generarea punctuală de cod, a fost valoros prin simplitatea și claritatea propunerilor oferite. Nu a fost un instrument de explorare conceptuală, ci mai degrabă unul de verificare și rafinare sintactică – o completare utilă în fazele de testare sau de comparare a soluțiilor alternative.

Windsurf, în schimb, a demonstrat un potențial ridicat pentru generarea automată a aplicațiilor, dar și o limitare fundamentală: lipsa de flexibilitate în absența unei structuri predefinite complet. Platforma devine eficientă doar atunci când dezvoltatorul furnizează diagrame UML clare, modele de date bine conturate și un set coerent de cerințe. În lipsa acestor elemente, aplicațiile generate sunt greu de adaptat sau extinse.

Instrumente precum CiteSpace au completat procesul de cercetare teoretică, oferind o perspectivă analitică avansată asupra tendințelor științifice. Totuși, complexitatea sa și cerințele metodologice ridicate îl transformă într-o unealtă de nișă, mai potrivită pentru cercetări de tip doctoral decât pentru proiecte software aplicate.

ClickUp s-a dovedit extrem de util în gestionarea proceselor și organizarea sarcinilor. Deși nu este un AI în sens strict, platforma folosește automatizări și corelări inteligente care contribuie semnificativ la eficiența echipei și claritatea proiectului. În combinație cu celelalte instrumente, ClickUp a oferit o bază solidă pentru managementul colaborativ al proiectului.

În mod notabil, DrawIO a jucat un rol esențial în proiectarea logică a aplicației, prin facilitarea modelării relaționale a datelor și a interacțiunilor dintre entități. Funcționalitățile sale semi-automate, asistate de AI, au accelerat procesul de diagramare și au contribuit la clarificarea arhitecturii sistemului.

În concluzie, adoptarea inteligenței artificiale în ingineria software trebuie realizată cu discernământ, pornind de la premisa că IA nu poate înlocui procesul ingineresc propriu-zis. Inteligența artificială nu înțelege contextul dacă nu este explicit furnizat, nu poate intuit nevoi neformulate și nu poate substitui expertiza umană în ceea ce privește analiza cerințelor, definirea arhitecturii sau validarea sistemului. Cu toate acestea, atunci

când este utilizată în complementaritate cu gândirea inginerescă, IA aduce beneficii incontestabile: accelerează procesul de dezvoltare, reduce volumul muncii repetitive și contribuie la creșterea calității documentației și a interfeței sistemului.

Prin urmare, cea mai eficientă abordare în utilizarea inteligenței artificiale rămâne cea colaborativă – în care dezvoltatorul definește cu rigoare ce trebuie construit, iar IA intervine pentru a automatiza, rafina și susține acest proces, fără a-i compromite fundamentele ingineresti.

REFERINȚE

- [1] S. Martínez-Fernández și colab., „Software Engineering for AI-Based Systems: A Survey”, *ACM Trans. Softw. Eng. Methodol.*, vol. 31, nr. 2, pp. 1–59, apr. 2022, doi: 10.1145/3487043.
- [2] Theodore Narku Odonkor, Nnaemeka Valentine Eziamaka, și Adetola Adewale Akinsulire, „Advancing financial inclusion and technological innovation through cutting-edge software engineering”, *Finance Account. Res. J.*, vol. 6, nr. 8, pp. 1320–1348, aug. 2024, doi: 10.51594/farj.v6i8.1375.
- [3] M. Barenkamp, J. Rebstadt, și O. Thomas, „Applications of AI in classical software engineering”, *AI Perspect.*, vol. 2, nr. 1, p. 1, dec. 2020, doi: 10.1186/s42467-020-00005-4.
- [4] C. Ebert și P. Louridas, „Generative AI for Software Practitioners”, *IEEE Softw.*, vol. 40, nr. 4, pp. 30–38, iul. 2023, doi: 10.1109/MS.2023.3265877.
- [5] Ruchit Parekh și Charles Smith, „Innovative AI-driven software for fire safety design: Implementation in vast open structure”, *World J. Adv. Eng. Technol. Sci.*, vol. 12, nr. 2, pp. 741–750, aug. 2024, doi: 10.30574/wjaets.2024.12.2.0345.
- [6] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Rickki, și D. Doermann, „Future of software development with generative AI”, *Autom. Softw. Eng.*, vol. 31, nr. 1, p. 26, mai 2024, doi: 10.1007/s10515-024-00426-z.
- [7] D. Russo, „Navigating the Complexity of Generative AI Adoption in Software Engineering”, *ACM Trans. Softw. Eng. Methodol.*, vol. 33, nr. 5, pp. 1–50, iun. 2024, doi: 10.1145/3652154.
- [8] Santhosh Bussa, „Evolution of Data Engineering in Modern Software Development”, *J. Sustain. Solut.*, vol. 1, nr. 4, pp. 116–130, dec. 2024, doi: 10.36676/j.sust.sol.v1.i4.43.